MODULE 19 · DOMAIN 2 - IMPLEMENT GENERATIVE AI AND AGENTIC SOLUTIONS

Agents III: multi-agent orchestration

When to add agents, handoff versus agent-as-tools, group chat and Magentic, Agent Framework orchestrations, and A2A in and out of Foundry.

DOMAIN 2 - Generative AI and agents WEIGHT 30-35% MODULE 19 of 39

admin@sthithapragna.com

Every extra agent is another **handoff that can fail**. Add one only for **specialisation**, a **security boundary**, or **parallel work**.

One agent or several?

SIGNAL	POINTS TO
Varied requests in one domain, a handful of tools	A single agent with tools
Instructions and tools too large to behave predictably	Several specialist agents
Parts of the work need different data access	Separate agents, each least-privilege
Independent analyses that could run at once	Concurrent specialist agents
Agents that would only rename a single step	Don't add them

ON THE EXAM

Microsoft calls a single agent with tools 'often the right default'. Multi-agent has to earn its overhead.

Five orchestration patterns and their shapes

PATTERN	SHAPE	WHO DECIDES WHAT'S NEXT
Sequential	A fixed pipeline	The workflow, in a set order
Concurrent	Fan-out, then fan-in	The workflow; results are aggregated
Group chat	A star, with a manager in the middle	The manager picks the next speaker
Handoff	A mesh, with no central orchestrator	Each agent decides when to hand off
Magentic	A manager with a task ledger	The manager plans, and replans

ON THE EXAM

Deterministic routing belongs in sequential or concurrent. Dynamic routing belongs in handoff, group chat or Magentic.

Handoff or agent-as-tools?

HANDOFF

Control moves

- One agent passes the whole task to another
- The receiving agent takes full ownership
- No central authority in charge
- Suits triage to a specialist who finishes the job

AGENT-AS-TOOLS

Control returns

- A primary agent calls others for subtasks
- Results come back to the primary agent
- The primary keeps overall responsibility
- Suits one assistant combining several experts

ON THE EXAM

Microsoft's A2A tool behaves like agent-as-tools: the called agent's answer goes back to the caller.

Who owns the conversation at the end? The **same agent**: agent-as-tools. Whoever it was **handed to**: handoff.

Managed collaboration

- A **manager** in the middle chooses who speaks next: round-robin, prompt-based or custom.
- Every participant sees the **full shared history**, so agents build on each other.
- Suits review, debate and **maker-checker** loops, with agents usually read-only.
- Set a **maximum number of turns** and a **termination condition**, or it can run indefinitely.
- Microsoft suggests **three or fewer** agents to keep control manageable.

> ON THE EXAM
>
> A group chat that never concludes needs a turn limit, a termination rule, and fewer voices.

A manager that plans the work

- Given a **goal**, a **manager** and **specialists**, the manager decides how the team works.
- It keeps a **task and progress ledger**, checking progress each round.
- When progress stalls, it can **reset and replan**.
- **Maximum** round and stall settings are the guardrails. It's slow, and cost is hard to predict.
- Suits open-ended problems, especially where a person should review the plan.

> ON THE EXAM
>
> No known plan, and a need for a reviewable plan: Magentic. A known plan: something simpler.

Agent Framework orchestrations

- Builders for all five patterns: **sequential, concurrent, handoff, group chat, Magentic**.
- All **stable in Python and .NET** since July 2026.
- All support **streaming, checkpointing, human approval**, and pause and resume.
- Builders produce ordinary workflows, so you can drop to a custom graph when needed.
- Run them inside a **hosted agent** to get Foundry identity, scaling and tracing.

> ON THE EXAM
>
> Orchestration logic lives in Agent Framework code. Foundry hosts it, secures it and observes it.

Orchestration builders in Agent Framework

```
from agent_framework.orchestrations import ConcurrentBuilder, SequentialBuilder

# Pipeline: each agent works on the previous agent's output
draft_review = SequentialBuilder(
    participants=[writer, legal_reviewer, editor]).build()

# Fan-out, fan-in: independent analyses of the same input, aggregated
due_diligence = ConcurrentBuilder(
    participants=[financial_analyst, legal_analyst, market_analyst]).build()
```

ON THE EXAM

Same agents, different builder, different behaviour. The pattern is a choice you make in one line.

Connected agents are gone

CLASSIC AGENT SERVICE

No longer available

- Connected Agents tool
- Main agent routing to sub-agents
- Not coming back to the new service

CURRENT REPLACEMENTS

Build with these

- A2A tool: call another agent, get the answer back
- Agent Framework orchestrations in a hosted agent
- Foundry workflows too, but they retire in December

ON THE EXAM

An answer naming connected agents describes the classic service, which retires on 31 March 2027.

Calling another agent

- Connect to any endpoint that speaks the **Agent2Agent (A2A)** protocol.
- A **project connection** stores the endpoint and its authentication.
- The remote agent's answer **returns to your agent**, which keeps managing the conversation.
- Works with other Foundry agents exposed as A2A endpoints, or agents built elsewhere.
- The remote agent's card is read anonymously, unless you set it to send credentials.

> ON THE EXAM
>
> The remote agent reasons for itself. Treat its answers as untrusted input, like any tool output.

Letting other agents call yours

- Enable incoming A2A, and Foundry **publishes an agent card** and accepts A2A requests.
- Requires the **responses protocol**. Prompt agents support it by default.
- Supports A2A **version 1.0 and 0.3**. New integrations should target **1.0**.
- The agent card needs **authentication**, at a custom path rather than the usual well-known path.
- Version 1.0 on Foundry's incoming endpoint is JSON-RPC only.

> ON THE EXAM
>
> A partner's agent in another framework or cloud calling yours is an incoming A2A scenario, not shared keys or roles.

MCP or A2A?

MCP

Agent to tools

- Exposes tools and data to an agent
- The tool does what it's asked, nothing more
- Your agent does all the reasoning

A2A

Agent to agent

- Connects an agent to another agent
- The other agent reasons and uses its own tools
- Discovered through an agent card

THE TIEBREAKER

Calling a capability: MCP. Delegating to something that thinks for itself: A2A.

What changes with several agents

CONCERN	WHAT TO DO
Context grows at every step	Pass summaries between agents, not full transcripts
Cost multiplies	Give each agent a model matched to its task
Data exposure	Security trimming in every agent, not just the first
Harmful content can enter mid-flow	Guardrails on tool calls and responses, not only output
Failures hide across agents	Trace every agent; child invoke_agent spans show the path
Loops and ping-pong handoffs	Turn limits and termination conditions

ON THE EXAM

Every row comes from Microsoft's orchestration guidance. Multi-agent problems are distributed-systems problems.

> A travel assistant talks to the customer throughout. It needs input from a flights specialist agent and a hotels specialist agent.
>
> The customer must receive one combined itinerary, from the same assistant they started with.

Which design fits?

A Handoff: the assistant transfers the customer to the flights agent.

B Agent-as-tools: the assistant calls both specialists and combines.

C Group chat: six agents discuss the trip until they reach agreement.

D Magentic: a manager builds a ledger and replans the trip each round.

◆ PAUSE AND DECIDE

Which design fits?

A. Handoff to flights.
Ownership moves to the flights agent, so the original assistant no longer combines anything.

B. Agent-as-tools.
Control returns to the assistant, which owns the conversation and synthesises one answer.

C. Six-agent group chat.
Overhead and loop risk, well beyond the three-agent guidance, for two known subtasks.

D. Magentic manager.
For open-ended problems. Flights plus hotels is a known plan.

THE SIGNAL

'Same assistant throughout' plus 'one combined answer' is agent-as-tools.

> A bank's classic Foundry solution uses the Connected Agents tool: an orchestrator routes questions to three specialist agents and uses their answers.
>
> The team is migrating to the new Foundry Agent Service, where the specialists will be prompt agents.

How should the orchestrator reach the specialists?

A Enable incoming A2A on the specialists; use the A2A tool.

B Turn on the Connected Agents tool for the new orchestrator agent.

C Rebuild the flow in the Foundry portal's visual workflow designer.

D Keep the classic solution on threads and runs, and revisit in 2028.

◆ PAUSE AND DECIDE

How should the orchestrator reach the specialists?

A Incoming A2A plus the A2A tool.

Microsoft's named replacement. Answers return to the orchestrator, as before.

B Connected Agents tool.

Not available in the new Agent Service, and not coming back.

C Visual workflow designer.

Retires on 1 December 2026, and hands off rather than returning answers.

D Stay classic until 2028.

Agents (classic) retire on 31 March 2027, well before then.

THE SIGNAL

Migrating connected agents where the orchestrator uses the answers: the A2A tool.

> A marketing team runs a group chat of five agents to review campaign plans, using a prompt-based manager to pick speakers.
>
> Sessions often run for over an hour without concluding, and token costs have tripled.

What should the team change?

A. Add a sixth agent whose instructions are to keep the others on topic.

B. Move every participant to a larger model so debates resolve sooner.

C. Switch to concurrent orchestration so all five agents reply at once.

D. Use three or fewer agents, and set a turn limit and termination rule.

◆ PAUSE AND DECIDE

What should the team change?

A. Add a sixth agent.
Another voice in a discussion that already can't conclude, with no hard stop.

B. Larger models.
Higher cost per turn, and still no rule for when to stop.

C. Concurrent orchestration.
Removes the review interaction the team wants, and needs a conflict strategy.

D. Three or fewer agents, turn limit, termination.
Microsoft's guidance: fewer voices, and a guaranteed end to every session.

THE TIEBREAKER

Group chats need a stopping rule in the orchestration, not a request in an agent's instructions.

> A logistics partner has built a booking agent in LangGraph, running in another cloud. It needs to ask your Foundry prompt agent for shipment status.
>
> Your security team won't share keys to your Foundry resource or give the partner roles in your project.

What should you do?

A. Give the partner your Foundry resource key, stored in their key vault.

B. Grant the partner's service principal the Foundry User role on your project.

C. Enable incoming A2A on your agent, targeting version 1.0 of the protocol.

D. Export your agent's definition as YAML for the partner to run themselves.

◆ PAUSE AND DECIDE

What should you do?

A Share the resource key.

Explicitly forbidden, and a resource key grants far more than one agent.

B Foundry User role for the partner.

Explicitly forbidden, and exposes the whole project, not one agent.

C Incoming A2A, version 1.0.

A standard, authenticated agent endpoint the partner's framework can call.

D Export the YAML.

Gives away the design, and the partner still can't reach your shipment data.

THE SIGNAL

An external agent, in another framework or cloud, calling yours: incoming A2A, targeting version 1.0.

Multi-agent orchestration in five lines

- **Add agents for specialisation, security boundaries or parallelism**, not by default.
- **Handoff moves ownership; agent-as-tools returns results.** Ask who owns the end.
- **Group chat:** three or fewer agents, turn limits, termination rules.
- **Connected agents are gone.** Use the A2A tool, or Agent Framework orchestrations.
- **MCP connects tools; A2A connects agents.** Incoming A2A: target version 1.0.

ON THE EXAM

Next, module 20: autonomous agents with safeguards, approval flows, agent monitoring and error analysis.